

RV32I 5-Stage Pipeline

Implementation Walkthrough of `src/`

How the single-cycle datapath was split into IF / ID / EX / MEM / WB, what each of the four pipeline registers carries, and why this build schedules its hazards in software instead of hardware.

Item	Detail
Scope of this document	The pipeline-register split only. Hazard detection, forwarding, stalling and flushing are deliberately not implemented yet.
New RTL	<code>src/</code> <code>Fetch_Cycle.v</code> , <code>Decode_Cycle.v</code> , <code>Execute_Cycle.v</code> , <code>Memory_Cycle.v</code> , <code>Writeback_Cycle.v</code> , <code>Pipeline_Top.v</code>
Reused unchanged	All eight functional units in <code>single_core/</code> - not one line edited
Verification	<code>src/</code> <code>Pipeline_Top_TestBench.v</code> - 12/12 self-checking assertions PASS
Companion document	<code>docs/</code> <code>RV32I_Single_Cycle_Core.pdf</code> (the single-cycle core)

Swastik Aditya Ranjan

B.Tech Electrical Engineering, NIT Rourkela

1. What This Document Covers

The single-cycle core in **single_core/** works and passes a 12-check regression. It has one structural problem: every instruction must complete fetch, decode, register read, ALU, memory access and write-back inside a single clock period. The clock can therefore be no faster than the slowest instruction's entire journey, and while the ALU is working the instruction memory sits idle, and vice versa. Most of the hardware is doing nothing most of the time.

Pipelining fixes the utilisation problem by cutting that journey into five shorter stages and letting five different instructions occupy them simultaneously. This document explains exactly how that cut was made in **src/**.

What you will find here

- The structure of the pipeline, and the naming convention that makes pipelined RTL readable (sections 3-4).
- A stage-by-stage tour of the five new modules (section 5).
- What each pipeline register carries, and - just as importantly - what it deliberately does not carry (section 6).
- The two backward signal paths, which are the entire source of difficulty in pipelining (section 7).
- Three design decisions that look odd out of context, and the reasoning behind each (section 8).
- Hazards: the arithmetic that produces the 3-NOP and 2-delay-slot rules, and an experiment confirming the hardware really behaves that way (section 9).

What is deliberately out of scope

This build has **no hazard detection unit, no forwarding network, no stalling and no flushing**. That is not an oversight or an unfinished corner - it is the defined scope of this stage of the project. Leaving the hazard hardware out makes the pipeline registers themselves, which are the actual subject of the work, easy to see and easy to reason about. The hazards are real and they are handled honestly, by scheduling NOPs in software and documenting precisely why each one is needed.

Read section 9 before concluding anything is broken. Every NOP in `src/program.hex` is there for a derived, measured reason.

2. What Changed, and What Deliberately Did Not

The most important fact about this implementation is how little of the existing design it touched.

Category	Files	Status
Functional units	PC.v, PC_Adder.v, instruction_Memory.v, Register_file.v, Sign_Extend.v, ALU.v, ALU_decoder.v, main_decoder.v, Control_Unit_Top.v, Data_Mem.v	Reused verbatim . Zero edits.
Single-cycle top	Single_Cycle_Top.v, its testbench and program.hex	Untouched and still passing.

Category	Files	Status
New pipeline RTL	Fetch_Cycle.v, Decode_Cycle.v, Execute_Cycle.v, Memory_Cycle.v, Writeback_Cycle.v, Pipeline_Top.v	New in src/.
New verification	src/program.hex, src/Pipeline_Top_TestBench.v	New in src/.

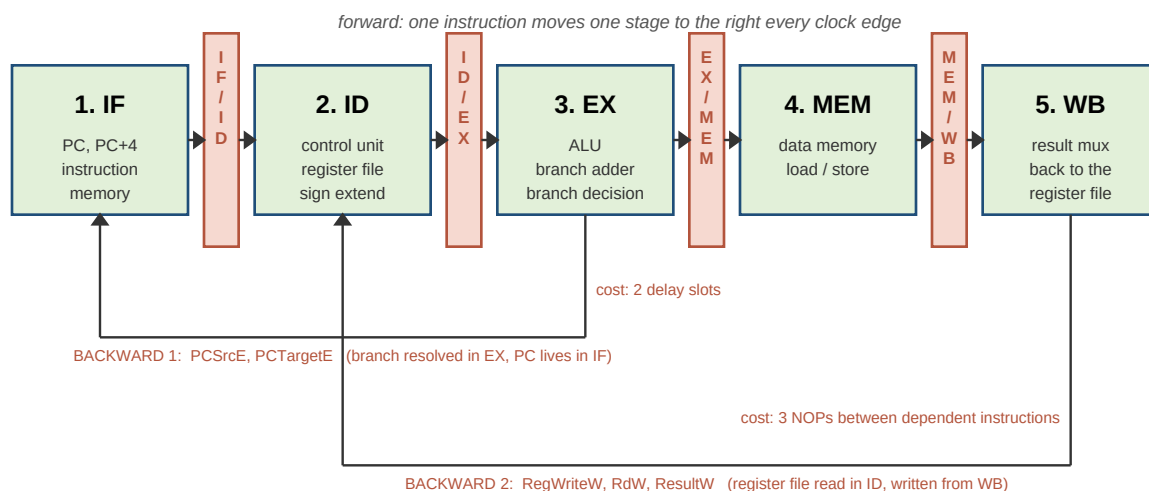
Why the functional units were shared rather than copied

Because **pipelining a processor does not change its functional units**. An ALU is an ALU. A register file is a register file. What pipelining changes is that you put registers *between* them and let several instructions be in flight at once. Sharing the modules makes that point structurally rather than just asserting it in prose.

There is also a practical debugging benefit. Because `single_core/` is frozen and demonstrably passing its own regression, any failure in the pipeline is provably a pipelining bug. If the shared modules had been forked and edited, every failure would start with the question "did I break the ALU, or did I wire the pipeline wrong?" That question never has to be asked here.

The cost of this choice shows up in section 8: two of the three design decisions documented there exist purely to avoid editing a verified file.

3. The Structure of the Pipeline



Every gate lives inside a stage. The four bars are the pipeline registers - the only thing this build adds.

Both backward paths are unprotected here: no forwarding, no stalling, no flushing.

Figure 1 - The 5-stage pipeline as implemented in `src/`. Green blocks are the five stage modules; the four salmon bars are the pipeline registers that give the design its name.

Read the figure left to right for the forward path: on every rising clock edge, each instruction advances exactly one stage. Once the pipe is full, five instructions are being worked on simultaneously and one

instruction completes per cycle.

The two paths drawn *below* the stages are the interesting part. They run backwards - right to left - and they are where all the difficulty in pipelining comes from. Section 7 covers them in detail.

The four pipeline registers

Each pipeline register is a bank of flip-flops written by one always @(posedge clk) block. There is nothing clever in any of them - they simply freeze their stage's results so the next stage can work on them next cycle while the current stage moves on to a new instruction.

Register	Written by	Purpose
IF/ID	bottom of Fetch_Cycle.v	Hands the fetched instruction word and its address to decode.
ID/EX	bottom of Decode_Cycle.v	Hands control bits, both register operands, the immediate, the PC and the destination register number to execute.
EX/MEM	bottom of Execute_Cycle.v	Hands the ALU result (the address, for a load or store), the store data and the surviving control bits to memory.
MEM/WB	bottom of Memory_Cycle.v	Hands both write-back candidates - ALU result and load data - to write-back.

Note that **Writeback_Cycle.v contains no pipeline register**. There is no sixth stage to hand anything to. Its output loops backwards to the register file instead, which is why WB is purely combinational.

Why non-blocking assignment is mandatory

Every pipeline register uses <=, never =. This is not a style preference.

All four registers update on the same clock edge. With non-blocking assignment, every one of them samples its *old* input and they all update together - which is exactly what real flip-flops do. With blocking assignment, the result would depend on the order the simulator happened to evaluate the four always blocks, and a value could race through two or three stages in a single cycle. The simulation would then disagree with the synthesised hardware, which is the worst possible class of bug.

4. The Naming Convention

Pipelined RTL is unreadable without a discipline for naming, because the same logical value exists in several places at once, at different ages. Every signal in src/ therefore ends in a letter naming the stage it lives in:

F = Fetch D = Decode E = Execute
M = Memory W = Write-back

PCF the program counter, as seen in the fetch stage
PCD the same value one cycle later, after the IF/ID register latched it
PCE the same value one cycle later again, after ID/EX latched it

The moment a signal crosses a pipeline register, its letter changes. That single habit is what makes this style of code navigable: if you see RD2E in one file and RD2M in another, you instantly know they are the same wire, one cycle apart, and you know which pipeline register connects them.

The convention also catches a whole class of bug by inspection. If an expression mixes suffixes - say it ANDs something ending in E with something ending in M - it is combining data from two different

instructions, and it is almost certainly wrong. There is exactly one place in this design where mixed suffixes are legitimate: the register file's write port in `Decode_Cycle.v`, which is driven by W-suffixed signals on purpose. That is the backward write-back path, and it is heavily commented for precisely this reason.

5. Stage-by-Stage Reference

5.1 Fetch - `src/Fetch_Cycle.v`

Reads the instruction the PC points at, and works out which address to read next. It never looks at the instruction it fetched - decoding is the next stage's job.

Contains `PC_Module`, `PC_Adder` (for `PC+4`), `instruction_Memory`, the PC-source mux, and the IF/ID register. The mux is the single-cycle core's mux unchanged, but note where its select signal comes from:

```
assign PCNextF = PCSrcE ? PCTargetE : PCPlus4F;
                ^^^^^^
                E suffix: produced two stages downstream
```

There is deliberately **no branch adder in this stage**, unlike the single-cycle core. The branch target is the branch's own address plus its immediate, and the immediate does not exist until `Sign_Extend` runs in decode. The adder therefore had to move downstream; section 8.2 explains why it landed in execute specifically.

5.2 Decode - `src/Decode_Cycle.v`

Turns the 32-bit instruction word into three things: control signals, the two register operands, and the sign-extended immediate. Contains `Control_Unit_Top`, `Register_file`, `Sign_Extend` and the ID/EX register.

This stage is where the single biggest conceptual shift from the single-cycle core happens:

Control signals are pipelined data. In the single-cycle core, the decoder's outputs went straight to the units that used them, because everything happened in one clock period. Here, `MemWrite` is decoded in ID but the data memory does not run until MEM, three cycles later. Wiring the decoder's `MemWrite` straight to the memory would make the memory obey whatever instruction happens to be sitting in decode at that moment - not the store that actually wants to write.

So control bits travel through the same pipeline registers as the data. The mental model worth carrying: *each instruction drags a little backpack of control bits down the pipe with it, and each stage reaches into the backpack for the bits it needs right now.* `RegWrite` has the longest journey of all - decoded in ID, used in WB, so it rides through three registers as `RegWriteE` -> `RegWriteM` -> `RegWriteW`.

The register file physically lives in this stage, but its *write* side is driven from write-back. That is why textbook diagrams draw the register file straddling ID and WB: it is one piece of hardware, read by one stage and written by another four cycles later.

5.3 Execute - `src/Execute_Cycle.v`

Two independent calculations happen here in parallel: the ALU does the instruction's real work, and a second adder computes the branch target. This stage also makes the only decision in the core that reaches backwards.

```
assign PCSrcE = BranchE & ZeroE;
```

```

          ^^^^^^  ^^^^^
          |         the ALU's zero flag, produced in THIS cycle
          |         a control bit that travelled one register from decode

```

Both terms describe the same instruction, so this AND is exactly the single-cycle branch equation - just evaluated one stage later. Section 8.1 explains how the two halves came to be split across two stages.

5.4 Memory - src/Memory_Cycle.v

Talks to the data memory, and nothing else. Exactly one instruction type reads it (lw), one writes it (sw), and every other instruction passes straight through untouched.

It is fair to ask why something most instructions ignore gets a whole stage. The answer is that a pipeline runs at the speed of its slowest stage, and every instruction must visit every stage in the same order. Memory access is among the slowest operations in the datapath, so it gets its own stage rather than being bolted onto the end of EX and dragging the whole clock period down. add pays a cycle of latency it does not need - but latency is not what a pipeline optimises. Throughput is, and one instruction still finishes every cycle.

Two values arrive here and it is worth keeping them straight: ALU_ResultM is the **address** (for lw/sw the ALU spent its cycle adding base register + offset), while WriteDataM is the **data** to store - the rs2 value, which bypassed the ALU completely back in execute.

5.5 Write-back - src/Writeback_Cycle.v

One 32-bit 2-to-1 mux. That is the entire stage.

```
assign ResultW = ResultSrcW ? ReadDataW : ALU_ResultW;
```

It decides only **what** to write. It does not decide *whether* - RegWriteW does that - nor *where* - RdW does that. Those two go straight from the MEM/WB register to the register file without passing through this module, because they need no further processing.

Why is the mux down here rather than back in MEM, where both inputs are already available? Because putting it in MEM would stack *memory read, then mux, then flip-flop* into a single clock period, on a stage that is already one of the slowest. Carrying both candidates through MEM/WB costs 32 extra flip-flops and buys a shorter critical path. That is a trade worth making, and it is the standard choice.

6. What Each Pipeline Register Carries

A pipeline register should carry only what a later stage genuinely still needs. Anything else is flip-flops and power spent for nothing. The table below tracks every signal across all four boundaries.

Signal	IF/ID	ID/EX	EX/MEM	MEM/WB	Consumed by
Instr	yes	-	-	-	the decoder, in ID
PC	yes	yes	-	-	the branch adder, in EX
RegWrite	-	yes	yes	yes	the register file, in WB
ResultSrc	-	yes	yes	yes	the result mux, in WB
MemWrite	-	yes	yes	-	the data memory, in MEM

Signal	IF/ID	ID/EX	EX/MEM	MEM/WB	Consumed by
ALUSrc	-	yes	-	-	the SrcB mux, in EX
Branch	-	yes	-	-	the PCSrcE AND gate, in EX
ALUControl	-	yes	-	-	the ALU, in EX
RD1 (rs1)	-	yes	-	-	the ALU, in EX
RD2 (rs2)	-	yes	yes	-	SrcB mux in EX; store data in MEM
ImmExt	-	yes	-	-	SrcB mux and branch adder, in EX
ALU_Result	-	-	yes	yes	memory address in MEM; result in WB
ReadData	-	-	-	yes	the result mux, in WB
Rd	-	yes	yes	yes	the register file write address, in WB

The bundle narrows as it goes

Look down the columns and you can watch signals die off as their last consumer runs. ImmExt, ALUControl, ALUSrc, Branch, RD1 and PC all stop at the EX/MEM boundary, because everything that needed them ran in execute. MemWrite stops at MEM/WB. That narrowing is normal and healthy.

Two entries deserve individual comment.

- **ImmSrc never enters a pipeline register at all.** It is a 2-bit selector needed only to *produce* ImmExt, which happens in decode. Once the 32-bit immediate exists, the selector has served its purpose and is thrown away. Only signals a later stage consumes deserve a seat.
- **PC+4 is not carried anywhere.** Textbook pipelines pipe it all the way to WB, because jal writes the return address into rd. This core does not implement jal, so PC+4 has exactly one consumer - the PC mux back in fetch - and never needs to leave that stage. Carrying it would be four registers of dead silicon. It goes in the moment jal does.

Note also that RD2 is renamed to WriteDataM as it crosses EX/MEM. Same wire, but the rename documents the role it is about to play: for a store, rs2 is the value being written to memory.

7. The Two Backward Paths

Forward data flow is the easy part of a pipeline - it is just registers in a row. Every genuine difficulty comes from the two paths that run *backwards*, because a backward path means a later stage is telling an earlier stage something it needed to know several cycles ago.

7.1 Branch redirect: EX to IF

Signals: PCSrcE, PCTargetE.

A branch is resolved in execute, but the PC lives in fetch. By the time the answer arrives, fetch has already read the two instructions that physically follow the branch in memory, and they are sitting in IF/ID and ID/EX. A complete pipeline kills them with flush logic. **This build has no flush logic, so those two instructions will execute.**

Consequence: two delay slots after every taken branch. `src/program.hex` puts NOPs there so that what executes is harmless. This is a real historical technique, not a hack invented to paper over a gap - early MIPS exposed the branch delay slot in its ISA for exactly this reason, and let compilers fill it with useful work.

A not-taken branch costs nothing: the two instructions after it were going to run anyway.

7.2 Register write-back: WB to ID

Signals: `RegWriteW`, `RdW`, `ResultW`.

The register file is read in decode but written from write-back, four stages later. Worse, `Register_file.v` writes on the rising edge and has no write-through bypass, so a read issued in the same cycle as a write returns the *old* value.

Consequence: three NOPs between a producer and its consumer. The arithmetic behind that number is derived in section 9.1 and confirmed experimentally in section 9.2.

8. Three Design Decisions Worth Defending

Three choices in `src/` look odd without context. Each is deliberate.

8.1 `Control_Unit_Top` is instantiated with `.zero(1'b1)`

In `Decode_Cycle.v` the control unit's zero input is tied to a constant one. This superficially resembles a genuine bug that once existed in the single-cycle core - documented in section 6 of the companion PDF - where the same port was hardcoded to `1'b0`. The two are opposites, and the difference matters.

`Control_Unit_Top` was written for a core where the ALU's zero flag was available in the same clock period as decoding. Internally it computes:

```
Branch (a.k.a. PCSrc) = <opcode is a branch> & zero
```

In a pipeline that equation cannot be evaluated in decode. The comparison producing the zero flag does not happen until the ALU runs in execute, one cycle later - the flag simply does not exist yet at that point in time.

What decode actually needs from the decoder is only the left half of that AND: the raw *"is this a branch instruction?"* bit. Tying zero to one extracts exactly that, because one is the identity element of AND:

```
BranchD = <opcode is a branch> & 1 = <opcode is a branch>
```

The other half is then performed in `Execute_Cycle.v`, where the real flag exists, as `PCSrcE = BranchE & ZeroE`. The logic is not weakened or discarded - it is **split across two stages**, which is the whole point of pipelining.

Contrast with the old defect: hardcoding zero to **0** was fatal because `x & 0 == 0` always, which destroyed the branch condition and made `beq` permanently not-taken. Hardcoding it to **1** destroys nothing and merely defers the real test by one stage.

The tidier alternative would be to add a clean `branch_op` output to `main_decoder`. That was rejected because it means editing `main_decoder.v`, `Control_Unit_Top.v` and `Single_Cycle_Top.v` - three files belonging to a core that currently passes its regression. The comment block in `Decode_Cycle.v` is roughly thirty lines long precisely because this line would otherwise look like the bug it is not.

8.2 The branch adder moved from fetch to execute

In the single-cycle core, Branch_Adder sat next to the PC. In a pipeline it cannot: the branch target is PC + immediate, and the immediate does not exist until Sign_Extend runs in decode. So the adder had to move at least as far down as ID.

It was placed in EX rather than ID for a routing reason. EX is where the branch condition is resolved, so keeping the target and the decision in the same stage means only **one** backward bundle has to be routed up to fetch, instead of two signals originating in two different stages. Fewer long backward wires is the right instinct in any pipeline.

The knock-on effect is that PC must be carried IF to ID to EX, purely so that execute has the branch's own address to add to. That is what the PC row in the section 6 table is recording.

8.3 The single_core/ modules are included via -I, not by relative path

The build requires an include search path:

```
cd src
iverilog -I ../single_core -o out.vvp Pipeline_Top_TestBench.v
vvp out.vvp
```

The obvious alternative - ``include "../single_core/Control_Unit_Top.v"` - does not work, and the failure is worth knowing about because it is not obvious. That file itself contains ``include "main_decoder.v"`, and **a nested include is resolved relative to the current working directory, not relative to the file doing the including**. So the outer include succeeds and the inner one fails with `"Include file main_decoder.v not found"`. The `-I` search path resolves correctly at every level of nesting.

9. Hazards, and Why This Build Solves Them in Software

A hazard is what happens when the pipeline's assumption - that instructions are independent - turns out to be false. This core has no hardware to detect them, so the test program is scheduled to avoid them. Every NOP in `src/program.hex` is derived below.

9.1 The data hazard, and where the number 3 comes from

Number each instruction by the slot it is fetched into. With one instruction issued per cycle, a 100 ns clock, and reset releasing before the first real rising edge:

```
instruction i occupies   IF   during (100i + 100, 100i + 200)
                        ID   during (100i + 200, 100i + 300)
                        EX   during (100i + 300, 100i + 400)
                        MEM   during (100i + 400, 100i + 500)
                        WB   during (100i + 500, 100i + 600)
```

Register_file.v commits its write on the rising edge that *ends* WB, and reads are latched into the ID/EX register on the edge that ends ID:

```
producer p commits its write at    t = 100p + 600
consumer c samples its reads at    t = 100c + 300
```

the consumer must sample strictly after the write lands:

```
100p + 600 < 100c + 300
c > p + 3
c >= p + 4
```

$c = p + 4$ means three instruction slots sit between producer and consumer. **Hence three NOPs.**

Why not two? Because Register_file.v has no write-through bypass. If the write were moved to the falling edge, or a read/write bypass added, this would shrink to two. Neither was done, because single_core/ is frozen and passing - see section 2.

Once forwarding is built, this gap collapses to **zero**, because the consumer grabs the value directly off the EX/MEM or MEM/WB wires instead of waiting for it to travel all the way to the register file and back.

9.2 Confirming the rule against real hardware

A derivation is a claim about the hardware, not a measurement of it. The rule was therefore tested: a scratch copy of the program was built with only **two** NOPs where the derivation demands three.

```
00500093    // slot 0    addi x1, x0, 5
00300113    // slot 1    addi x2, x0, 3
00000013    // slot 2    nop
00000013    // slot 3    nop          <-- only two NOPs, not three
002081b3    // slot 4    add  x3, x1, x2

result:    ok   : x1 = 5
           ok   : x2 = 3
           FAIL: x3 = 5 (expected 8)
```

The failure is not merely present, it is *exactly the failure the arithmetic predicts*, which is much stronger evidence than a bare pass/fail:

- x1 is produced in slot 0, consumed in slot 4. $4 \geq 0 + 4$ holds, so x1 reads correctly as 5.
- x2 is produced in slot 1, consumed in slot 4. $4 \geq 1 + 4$ fails, so x2 still reads as its reset value 0.
- The ALU therefore computes $5 + 0 = 5$, and x3 lands on 5 rather than 8. Precisely the observed value.

One instruction being correct while the other is stale, in the same instruction, is a signature that no amount of hand-waving produces by accident. The 3-NOP rule describes this hardware exactly.

9.3 The control hazard, and where the number 2 comes from

The branch at slot i resolves during its EX interval, so the PC only accepts PCTargetE at $t = 100i + 400$. Fetch never stopped: slots $i+1$ and $i+2$ have already been read from memory. With no flush logic, both execute. **Hence two delay slots.**

This is why the taken branch in src/program.hex uses an offset of +16 where the single-cycle version used +8 - the two delay slots sit between the branch and the instruction being skipped:

```
0x3c    beq x1, x1, +16    the branch
0x40    nop              delay slot 1 - runs anyway
0x44    nop              delay slot 2 - runs anyway
0x48    addi x11, x0, 99   the instruction we actually want to skip
0x4c    addi x12, x0, 7    target: 0x4c - 0x3c = 16
```

The encoding was verified by hand against Sign_Extend.v's B-type unscrambling $\{\{19\{In[31]\}\}, In[31], In[7], In[30:25], In[11:8], 1'b0\}$. For $0x00108863$: $In[31]=0$, $In[7]=1$, $In[30:25]=000000$, $In[11:8]=0000$, giving $0_1_000000_0000_0 = 16$. Correct.

9.4 The hazard that needs no NOP at all

It would be easy to assume a store followed immediately by a load from the same address also needs padding. It does not, and the reason is worth following because it is genuinely tight.

Data_Memory writes on the rising clock edge but reads combinationaly. The store at slot 11 commits its write at the edge that *ends* its MEM interval, $t = 1600$. The load at slot 12 has its MEM interval running from $t = 1600$ to $t = 1700$, and its read is combinational across that whole window - so it sees memory *after* the store landed. Its result is latched into MEM/WB at $t = 1700$.

So `sw` immediately followed by `lw` works with zero NOPs. It works only because the write happens at the boundary and the read happens across the interval that follows. The regression exercises this deliberately: `x9 = 8` in the check list is what proves it.

10. The Test Program

`src/program.hex` computes exactly the same twelve results, into the same registers, as `single_core/program.hex`. That equivalence is deliberate: the two cores must reach an **identical architectural outcome**, so the same twelve assertions can be used on both. What differs is the schedule.

idx	addr	encoding	instruction	why it is here
0	0x00	00500093	<code>addi x1, x0, 5</code>	<code>x1 = 5</code>
1	0x04	00300113	<code>addi x2, x0, 3</code>	<code>x2 = 3</code>
2	0x08	00000013	<code>nop</code>	\
3	0x0c	00000013	<code>nop</code>	> 3 NOPs: let <code>x1</code> and <code>x2</code> commit
4	0x10	00000013	<code>nop</code>	/
5	0x14	002081b3	<code>add x3, x1, x2</code>	<code>x3 = 8</code> ($5 \geq 0+4$ ok, $5 \geq 1+4$ ok)
6	0x18	40208233	<code>sub x4, x1, x2</code>	<code>x4 = 2</code>
7	0x1c	0020f2b3	<code>and x5, x1, x2</code>	<code>x5 = 1</code>
8	0x20	0020e333	<code>or x6, x1, x2</code>	<code>x6 = 7</code>
9	0x24	001123b3	<code>slt x7, x2, x1</code>	<code>x7 = 1</code>
10	0x28	0020a433	<code>slt x8, x1, x2</code>	<code>x8 = 0</code>
11	0x2c	00302023	<code>sw x3, 0(x0)</code>	<code>mem[0] = 8</code> . <code>x3</code> from idx 5:
				11 $\geq 5+4$, so the six
				independent ops above double
				as the required spacing
12	0x30	00002483	<code>lw x9, 0(x0)</code>	<code>x9 = 8</code> . No NOP needed - see 9.4
13	0x34	00208463	<code>beq x1, x2, +8</code>	NOT taken ($5 \neq 3$), no slots needed
14	0x38	00100513	<code>addi x10, x0, 1</code>	<code>x10 = 1</code> - proves fall-through
15	0x3c	00108863	<code>beq x1, x1, +16</code>	TAKEN ($5 == 5$) -> 0x4c
16	0x40	00000013	<code>nop</code>	delay slot 1
17	0x44	00000013	<code>nop</code>	delay slot 2
18	0x48	06300593	<code>addi x11, x0, 99</code>	MUST be skipped -> <code>x11</code> stays 0
19	0x4c	00700613	<code>addi x12, x0, 7</code>	<code>x12 = 7</code> - the branch target

Notice that the six independent R-type instructions at slots 5-10 are doing double duty: they are the instructions under test, *and* they provide the spacing the store at slot 11 needs before it can read `x3`. Real schedulers do this constantly - filling hazard gaps with useful work rather than NOPs is most of what an instruction scheduler is for.

Five of the twenty words are NOPs. That is a 25% overhead, and it is the honest price of having no hazard hardware. Forwarding would eliminate the three data-hazard NOPs entirely; flush logic would eliminate the two delay slots.

Why `x11` is the most important check

`x11` is never written by this program, and reads back as 0 because `Register_file.v` clears all 32 registers on reset. If the branch silently failed to redirect the PC, slot 18 would run and `x11` would become 99. Together with `x12 = 7`, that pair is what makes this a real test of branch behaviour rather than a test that

passes by accident - a check that can only pass for the right reason.

11. Verification

```
cd src
iverilog -I ../single_core -o out.vvp Pipeline_Top_TestBench.v
vvp out.vvp
gtkwave Pipeline_Top_TestBench.vcd          # optional
```

The testbench is self-checking - it peeks into the register file hierarchically and asserts twelve expected values, rather than relying on a manual waveform read:

```
=== 5-stage pipelined RV32I regression (src/program.hex) ===
ok  : x1 = 5
ok  : x2 = 3
ok  : x3 = 8
ok  : x4 = 2
ok  : x5 = 1
ok  : x6 = 7
ok  : x7 = 1
ok  : x8 = 0
ok  : x9 = 8
ok  : x10 = 1
ok  : x11 = 0
ok  : x12 = 7
RESULT: PASS - all 12 checks passed
```

The hierarchical path is one level deeper than in the single-cycle testbench - `DUT.Decode.Register_file.Register[n]` - because in a pipeline the register file belongs to the decode stage, even though write-back drives its write port.

`vvp` also prints a `$readmemh`: Not enough words in the file for the requested range warning. Expected and harmless: `program.hex` is 20 words, instruction memory is 1024, and the remainder was already zero-filled - which decodes to a harmless NOP.

How long the run takes, and why

The last instruction to retire is `addi x12, x0, 7` at `0x4c`. It is **fetch slot 18, not 19**: the taken branch at slot 15 redirects the PC at `t = 1900`, so slot 18 fetches `0x4c` and address `0x48` is never fetched at all. Its write commits at `t = 600 + 1800 = 2400`, and the testbench samples at `t = 2625`.

Two costs versus the single-cycle core, both structural rather than accidental:

- **Pipeline fill.** Nothing retires until `t = 600` - four dead cycles while the first instruction walks down the pipe.
- **The NOPs.** Five of the twenty words exist purely to space out hazards, and each still costs a full cycle.

Neither is a throughput loss. Once the pipe is full, one instruction still completes every cycle - which is the entire point of the exercise.

12. What Is Not Implemented Yet

Stated plainly, so nothing here is mistaken for a claim it does not make:

Not implemented	What it would do	What it would remove
Forwarding / bypass network	Route EX/MEM and MEM/WB results directly back to the ALU inputs, so a consumer never waits for the register file.	All three data-hazard NOPs.
Hazard detection unit	Compare RdE/RdM/RdW against the rs1/rs2 fields in decode and drive the forwarding muxes.	The need to hand-schedule dependent instructions.
Load-use stall	Detect the one case forwarding cannot fix - a load feeding the very next instruction - and hold IF/ID and ID/EX for one cycle.	The remaining correctness gap after forwarding.
Branch flush	Clear IF/ID and ID/EX when PCSrcE asserts, turning the two wrongly-fetched instructions into bubbles.	Both delay-slot NOPs.

These are the next stage of the project, and the order above is roughly the order they should be built in: forwarding first, because it removes the most NOPs for the least logic; then the load-use stall, because it is the one case forwarding provably cannot solve; then flushing.

A useful property of the current build is that it gives all four features a **regression to be measured against**. When forwarding lands, the three data-hazard NOPs can be deleted from program.hex and the same twelve assertions must still pass. When flushing lands, the two delay slots can go and the branch offset returns to +8. Each feature has a concrete, falsifiable definition of done, expressed as NOPs removed while the twelve results stay identical.

Appendix A. File Map

File	Contains	Stage
src/Fetch_Cycle.v	PC_Module, PC_Adder, instruction_Memory, PC-source mux, IF/ID register	1 - IF
src/Decode_Cycle.v	Control_Unit_Top, Register_file, Sign_Extend, ID/EX register	2 - ID
src/Execute_Cycle.v	ALU, Branch_Adder, SrcB mux, PCSrcE AND gate, EX/MEM register	3 - EX
src/Memory_Cycle.v	Data_Memory, MEM/WB register	4 - MEM
src/Writeback_Cycle.v	Result mux only - no pipeline register	5 - WB
src/Pipeline_Top.v	Wiring and includes only. No logic.	all
src/program.hex	20-instruction regression program with derived NOP scheduling	-
src/Pipeline_Top_TestBen ch.v	Self-checking regression, 12 assertions	-

Appendix B. Signal Reference

Signal	Width	Meaning
PCF / PCD / PCE	32	The instruction's own address, as it travels IF, ID, EX. Needed in EX by the branch adder.

Signal	Width	Meaning
PCPlus4F	32	PCF + 4. Never leaves the fetch stage - see section 6.
PCNextF	32	Output of the PC-source mux; the address fetched next cycle.
InstrF / InstrD	32	The instruction word. Dies at the ID/EX boundary, fully translated into control bits and operands.
PCSrcE	1	Backward path 1. BranchE & ZeroE - 1 means a branch in EX resolved as taken.
PCTargetE	32	Backward path 1. PCE + ImmExtE, where a taken branch wants to go.
BranchE	1	Raw branch-opcode bit. Not "branch taken" - see section 8.1.
ZeroE	1	ALU zero flag. For beq the ALU subtracts, so this is high exactly when rs1 == rs2.
RD1E / RD2E	32	The rs1 and rs2 register values read in decode.
ImmExtE	32	Sign-extended immediate. Feeds both the SrcB mux and the branch adder, both in EX.
SrcBE	32	Second ALU operand after the ALUSrc mux: rs2 or the immediate.
ALU_ResultM	32	The address for a load or store; the answer for everything else.
WriteDataM	32	rs2, renamed at the EX/MEM boundary to document its role as store data.
ReadDataW	32	The word a load pulled out of data memory.
RegWriteW	1	Backward path 2. Register file write enable. The longest journey in the design: decoded in ID, used in WB.
RdW	5	Backward path 2. Destination register number.
ResultW	32	Backward path 2. The value written into RdW.